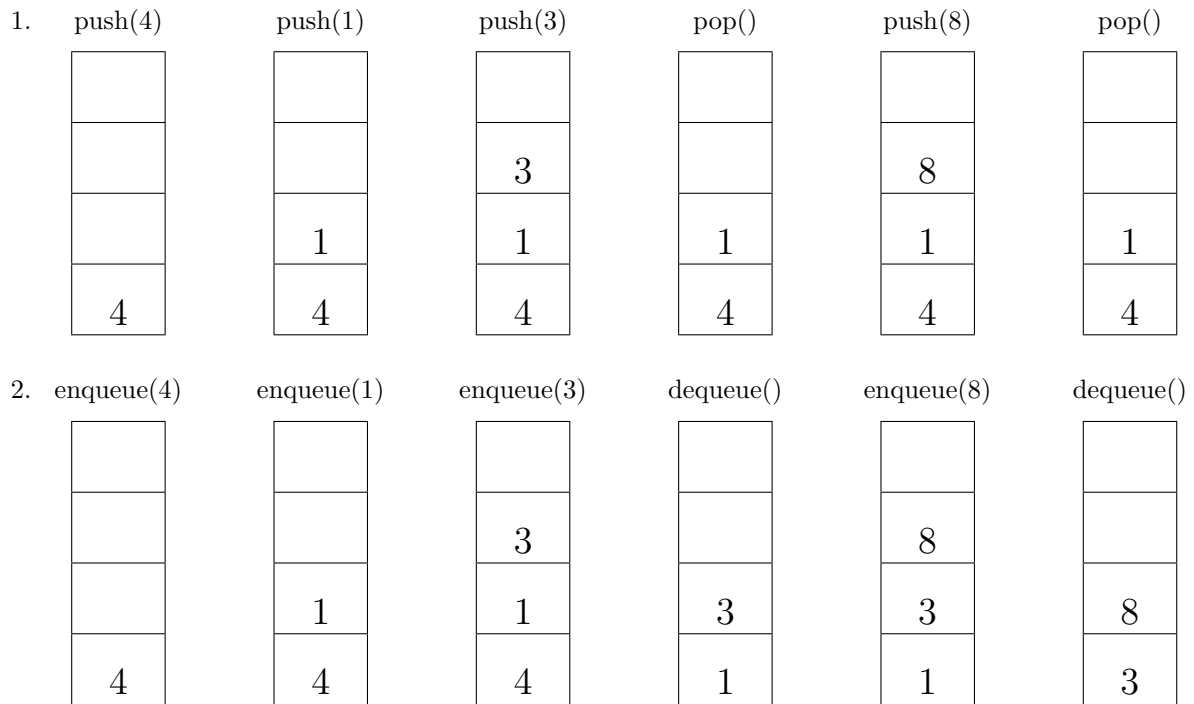


Informatics II, Spring 2026, Solution 7

Task 1



3. The first stack starts at 1 and grows up towards n , while the second starts from n and decreases to 1. Stack overflow happens when an element is pushed when the two stack pointers are adjacent.

4. ENQUEUE: $\Theta(1)$.

DEQUEUE: worst $O(n)$, amortized $\Theta(1)$ (on average).

Let the two stacks be A and B .

ENQUEUE pushes elements on B . ENQUEUE is always $\Theta(1)$. DEQUEUE pops elements from A . If A is empty, the contents of B are transferred to A by popping them one by one out of B and pushing them to A until B is empty. That way, they appear in reverse order and are popped in the original order.

DEQUEUE operation can perform in $\Theta(n)$ time, but that will happen only when A is empty. If many ENQUEUES and DEQUEUES are performed, the total time will be linear to the number of elements. For example, we ENQUEUE n elements and DEQUEUE n elements. All n elements are transferred from B to A only once, so in total n times. The amortized complexity of DEQUEUE = $n / n = 1$, which is $\Theta(1)$ (on average).

5. PUSH: $\Theta(1)$.

POP: $\Theta(n)$.

We have two queues – q_1 and q_2 . PUSH operation always enqueues elements in q_1 . Assume that q_1 contains i elements: $e_1, \dots, e_i$. POP operation: (1) dequeue $e_1, \dots, e_{i-1}$ elements and remain element e in q_1 (2) enqueue $e_1, \dots, e_{i-1}$ in order to q_2 . (3) dequeue e from q_1 and return e .

The PUSH operation is $\Theta(1)$. The POP operation is $\Theta(n)$ where n is the number of elements in the stack. In other words, there are n elements in q_1 .

Task 2.1

```
1 #include <stdio.h>
2 #define SIZE 10
3
4 int stack[SIZE];
5 int top = -1;
6
7 void push(int value){
8     if(top<SIZE-1){
9         if (top < 0){
10             stack[0] = value;
11             top = 0;
12         }
13         else{
14             stack[top+1] = value;
15             top++;
16         }
17     }
18     else{
19         printf("Stackoverflow!!!\n");
20     }
21 }
22
23 int isempty() {
24     return top<0;
25 }
26
27 int pop(){
28     if(!isempty()){
29         int n = stack[top];
30         top--;
31         return n;
32     }
33     else{
34         printf("Error: the stack is empty!\n");
35         return -99999;
36     }
37 }
38
39 int Top(){
```

```
40     if (!isempty()){
41         return stack[top];
42     }
43     else{
44         printf("Error: the stack is empty!\n");
45         return -99999;
46     }
47 }
48
49 void display(){
50     int i;
51     for(i=0;i<=top;i++){
52         printf("%d,",stack[i]);
53     }
54     printf("\n");
55 }
56
57 int main(){
58     push(4);
59     push(8);
60     printf("isempty: %d\n", isempty());
61     printf("Top: %d\n", Top());
62     display();
63
64     pop();
65     printf("\nisempty: %d\n", isempty());
66     printf("Top: %d\n", Top());
67     display();
68
69     pop();
70     printf("\nisempty: %d\n", isempty());
71     printf("Top: %d\n", Top());
72     display();
73
74     pop();
75
76     return 0;
77 }
```

Task 2.2

```
1 #include <stdio.h>
2 #define MAXSIZE 10
3
4 int queue[MAXSIZE];
5
6 int front = -1;
7 int rear = -1;
8 int size = -1;
9
10 int isempty()
11 {
12     return size <= 0;
13 }
14
15 int isfull()
16 {
17     return size == MAXSIZE;
18 }
19
20 void enqueue(int value)
21 {
22     if(size < MAXSIZE)
23     {
24         if(isempty())
25         {
26             queue[0] = value;
27             front = rear = 0;
28             size = 1;
29         }
30         else if(rear == MAXSIZE-1)
31         {
32             queue[0] = value;
33             rear = 0;
34             size++;
35         }
36         else
37         {
38             queue[rear+1] = value;
39             rear++;
40             size++;
41         }
42     }
43     else
44     {
45         printf("Queue is full\n");
46     }
47 }
48
49 int Front()
50 {
51     if(isempty())
52     {
53         printf("Queue is empty\n");
54         return -1;
55     }
56     else
57     {
58         return queue[front];
59     }
60 }
61
62 int dequeue()
63 {
64     int ret = Front();
65     size--;
66     front++;
67     if (front == MAXSIZE) {
68         front = 0;
69     }
70     return ret;
71 }
72
73 void display()
74 {
75     if(isempty())
76     {
77         printf("Queue is empty\n");
78         return;
79     }
80
81     int i;
82     if(rear >= front)
83     {
84         for(i=front; i<=rear; i++)
85         {
86             printf("%d,", queue[i]);
87         }
88     }
89     else
90     {
91         for(i=front; i<MAXSIZE; i++)
92         {
93             printf("%d,", queue[i]);
94         }
95         for(i=0; i<=rear; i++)
96         {
97             printf("%d,", queue[i]);
98         }
99     }
100     printf("\n");
101 }
102
103 int main()
104 {
105     display();
106     enqueue(4);
107     enqueue(8);
108     enqueue(10);
109     enqueue(20);
110     display();
111     dequeue();
112     printf("After dequeue\n");
113     display();
114     enqueue(50);
115     enqueue(60);
116     enqueue(70);
117     enqueue(80);
118     dequeue();
119     enqueue(90);
120     enqueue(100);
121     enqueue(110);
122     enqueue(120);
123     enqueue(130);
124     enqueue(140);
125     enqueue(150);
126     printf("After enqueue\n");
127     display();
128     dequeue();
129     printf("After dequeue\n");
130     display();
131     enqueue(160);
132     printf("After enqueue\n");
133     display();
134     return 0;
135 }
```

Task 2.3

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define TRUE 1
4 #define FALSE 0
5
6 struct node
7 {
8     int data;
9     struct node *next;
10 };
11 typedef struct node node;
12
13 node *top;
14
15 void initialize()
16 {
17     top = NULL;
18 }
19
20 void push(int value)
21 {
22     node *tmp;
23     tmp = malloc(sizeof(node));
24     tmp->data = value;
25     tmp->next = top;
26     top = tmp;
27 }
28
29 int pop()
30 {
31     node *tmp;
32     int n;
33     tmp = top;
34     n = tmp->data;
35     top = tmp->next;
36     free(tmp);
37     return n;
38 }
```

```
39
40 int Top()
41 {
42     return top->data;
43 }
44
45 int isempty()
46 {
47     return top==NULL;
48 }
49
50 void display(node *head)
51 {
52     if(head == NULL)
53     {
54         printf("NULL\n");
55     }
56     else
57     {
58         printf("%d,", head->data);
59         display(head->next);
60     }
61 }
62
63 int main()
64 {
65     initialize();
66     push(10);
67     push(20);
68     push(30);
69     printf("The_top_is_%d\n", Top());
70     pop();
71     printf("The_top_after_pop_is_%d\n", Top());
72     display(top);
73     return 0;
74 }
```

Task 2.4

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #define TRUE 1
4 #define FALSE 0
5 #define FULL 10
6
7 struct node
8 {
9     int data;
10    struct node *next;
11 };
12 typedef struct node node;
13
14 struct queue
15 {
16     int count;
17     node *front;
18     node *rear;
19 };
20 typedef struct queue queue;
21
22 void initialize(queue *q)
23 {
24     q->count = 0;
25     q->front = NULL;
26     q->rear = NULL;
27 }
28
29 int isempty(queue *q)
30 {
31     return (q->rear == NULL);
32 }
33
34 void enqueue(queue *q, int value)
35 {
36     if (q->count < FULL)
37     {
38         node *tmp;
39         tmp = malloc(sizeof(node));
40         tmp->data = value;
41         tmp->next = NULL;
42         if(!isempty(q))
43         {
44             q->rear->next = tmp;
45             q->rear = tmp;
46         }
47         else
48         {
49             q->front = q->rear = tmp;
50         }
51     }
```

```
51         q->count++;
52     }
53     else
54     {
55         printf("List is full\n");
56     }
57 }
58
59 int dequeue(queue *q)
60 {
61     node *tmp;
62     int n = q->front->data;
63     tmp = q->front;
64     q->front = q->front->next;
65     if(q->front == NULL)
66         q->rear = NULL;
67     q->count--;
68     free(tmp);
69     return(n);
70 }
71
72 void display(node *head)
73 {
74     if(head == NULL)
75     {
76         printf("NULL\n");
77     }
78     else
79     {
80         printf("%d,", head->data);
81         display(head->next);
82     }
83 }
84
85 int main()
86 {
87     queue *q;
88     q = malloc(sizeof(queue));
89     initialize(q);
90     enqueue(q,10);
91     enqueue(q,20);
92     enqueue(q,30);
93     printf("Queue,before_dequeue\n");
94     display(q->front);
95     dequeue(q);
96     printf("Queue,after_dequeue\n");
97     display(q->front);
98     return 0;
99 }
```

Task 3.

Algorithm: REVERSEEVEN(Q)

```
S = initStack()
qSize = queueSize(Q)
for  $i = 1$  to  $qSize$  do
    val = deQueue(Q)
    if  $val \% 2 == 0$  then
        | push(S, val)
    | enqueue(Q, val)
for  $i = 1$  to  $qSize$  do
    val = deQueue(Q)
    if  $val \% 2 == 0$  then
        | enqueue(Q, pop(S))
    else
        | enqueue(Q, val)
```